

DSAA Lab 09

Implementation of search, insert, and delete operations on a (BST)

Areeba (TCT-004)

Thar Institute of Engineering Sciences and Technology (TIEST)

CT-159: Data Structure Algorithms and Applications (Pr)

Engr. Aneel Kumar

October 30, 2025

Exercise

Q1: Analyze the given code to find the difference between “find” and “search” methods.

Answer:

Feature	search ()	FindMin ()
Purpose	Used to locate a specific value (key) in the Binary Search Tree (BST).	Used to find the smallest (minimum) value in the entire subtree.
Input	Takes a node and a data value to be searched.	Takes only a node (usually the root of a subtree).
Working	Compares the given value with the node's data. If smaller, moves left; if larger, moves right; continues until it finds the node or reaches null.	Keeps traversing the left child repeatedly until it reaches the leftmost node, which holds the minimum value.
Return Type	Returns the node that contains the searched value, or null if not found.	Returns the node with the minimum data value.
Usage Example	Used to check if a value exists in the tree.	Used during deletion , to find the inorder successor (smallest node in the right subtree).

Q2: Merge “find” and “search” methods into a single method by necessary modifications in the given code and execute it.

Answer:

I merged find and search methods into a single method called searchOrFind

```
import java.util.*;

class Node {
    int data; Node left, right;
    Node(int d) { data = d; }
}

public class BinaryTree {
    Node root;

    Node insert(Node r, int d) {
        if (r == null) return new Node(d);
        if (d < r.data) r.left = insert(r.left, d);
        else if (d > r.data) r.right = insert(r.right, d);
        return r;
    }

    void searchOrFind(Node r, int key) {
        if (r == null) System.out.println("Key not found!");
        else if (r.data == key) System.out.println("Key found!");
        else if (key < r.data) searchOrFind(r.left, key);
        else searchOrFind(r.right, key);
    }

    void inorder(Node r) {
        if (r != null) { inorder(r.left); System.out.print(r.data + " "); inorder(r.right); }
    }
}
```

```
}  
  
public static void main(String[] a) {  
    Scanner sc = new Scanner(System.in);  
    BinaryTree t = new BinaryTree();  
    System.out.print("Enter number of nodes: ");  
    int n = sc.nextInt();  
    System.out.println("Enter values:");  
    for (int i = 0; i < n; i++) t.root = t.insert(t.root, sc.nextInt());  
    System.out.print("Enter key to search: ");  
    t.searchOrFind(t.root, sc.nextInt());  
    System.out.print("Inorder: "); t.inorder(t.root);  
}  
}
```